MODULE 4 · NETWORKING

Endpoints & Headless Services

How a Service tracks its Pods — and what happens when it skips the middleman

WHY IT MATTERS

A Service IP is a promise — EndpointSlice is the receipt

Every Service is backed by an **EndpointSlice** object listing the real Pod IPs it load-balances to — the legacy **Endpoints** (v1) API still exists but is being phased out. That list changes constantly as Pods restart, scale, or fail readiness.

Sometimes clients don't want a single virtual IP — they want the **Pod IPs themselves**. That's what a **headless Service** gives you.

BY THE END YOU CAN

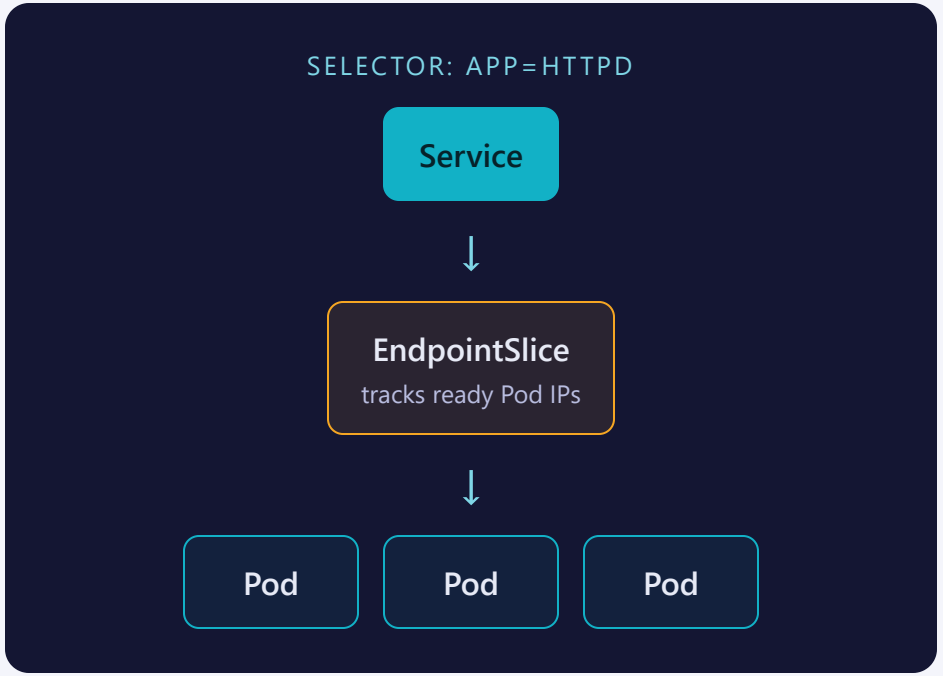
- Explain how Endpoints/EndpointSlices back a Service
- Set `clusterIP: None` and know what changes
- Read the DNS difference: one IP vs. per-Pod A records
- Know when a headless Service belongs — StatefulSets

CONCEPT

Same tracking, two different jobs at the end

Every Service — headless or not — watches Pods matching its `selector` and writes their ready IPs into an **EndpointSlice**.

A normal Service then puts a **virtual ClusterIP** in front of that list and load-balances. A headless Service (`clusterIP: None`) skips the virtual IP entirely — DNS hands clients the Pod IPs directly.



ENDPOINTS VS. ENDPOINTSLICE

Same data, a newer, more scalable shape

OBJECT	SHAPE	STATUS
`Endpoints`	One object per Service, all IPs in one list	Legacy API — still readable, no longer the source of truth
`EndpointSlice`	Sharded — multiple slices per Service at scale (~100 IPs each)	What `kube-proxy` and CoreDNS actually consume

SAME BACKING PODS EITHER WAY

Headless and ClusterIP Services both populate Endpoints/EndpointSlices identically. The difference is entirely in what the Service **does** with that list.

MANIFEST ANATOMY

One field flips the behaviour

```yaml
# httpd-svc — normal
apiVersion: v1
kind: Service
metadata:
  name: httpd-svc
  namespace: training
spec:
  type: ClusterIP
  selector:
    app: httpd
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

```yaml
# httpd-headless
apiVersion: v1
kind: Service
metadata:
  name: httpd-headless
  namespace: training
spec:
  clusterIP: None
  selector:
    app: httpd
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
```

type: ClusterIP → **clusterIP: None** is the only line that changes — `selector`, `ports`, and metadata stay identical, and both still populate Endpoints/EndpointSlices. Note it's not the same as omitting the field: omitting it still auto-assigns a ClusterIP.

DO IT — TWO WAYS

Headless has no imperative shortcut

IMPERATIVE	DECLARATIVE
`kubectl expose deploy httpd-deploy \ --port=80` → always assigns a ClusterIP	`kubectl apply -f headless-service.yaml`

THE BRIDGE TRICK

```
$ kubectl expose deploy httpd-deploy --port=80 \
    --dry-run=client -o yaml > svc.yaml   # then hand-edit clusterIP: None
```

SEE IT

DNS is where the difference shows up

`get svc` shows the only visible difference: a real ClusterIP vs. None. Both services list the **same** Pod IPs under `get endpointslices`.

Query DNS and the behaviour splits: the normal Service resolves to **one** IP; the headless Service resolves to **every** Pod IP as separate A records.

```
$ nslookup httpd-svc.training.svc.cluster.local
Address: 10.96.12.44        # the ClusterIP

$ nslookup httpd-headless.training.svc.cluster.local
Address: 10.1.1.4           # Pod A
Address: 10.1.1.5           # Pod B
Address: 10.1.1.6           # Pod C
```

WHEN TO USE HEADLESS

Skip load-balancing when identity matters more

Load-balancing hides *which* Pod answered — fine for stateless replicas, wrong when a client needs a **specific** Pod.

A **StatefulSet** pairs with a headless Service to give each Pod a stable, resolvable DNS name: `pod-0.svc.ns.svc.cluster.local`.

GOOD FIT

- StatefulSets — stable per-Pod DNS identity
- Client-side load balancing (client picks the IP)
- Peer discovery between replicas (databases, clusters)

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Expecting a ClusterIP.** Headless Services always show `None` — that's correct, not broken.
- **Assuming load balancing still happens.** Headless DNS returns every Pod IP; the client decides how to spread requests.
- **Not-ready Pods still tracked separately.** `describe endpoints` splits ready vs. not-ready — not-ready Pods aren't served.
- **Forgetting the selector.** No matching Pods → empty EndpointSlice, headless or not.

YOUR ENDPOINTS TOOLKIT

Commands to keep at hand

```
$ kubectl get svc -n NS                      # ClusterIP vs None
$ kubectl get endpointslices -n NS           # which Pod IPs are wired in — the sharded, modern view
$ kubectl get endpoints NAME -n NS           # legacy, still populated
$ kubectl describe endpoints NAME -n NS      # ready vs not-ready split
$ nslookup SVC.NS.svc.cluster.local           # one IP, or one per Pod?
```

Tip: when in doubt whether a Service is headless, `get svc` and check the `CLUSTER-IP` column for `None`.

RECAP

Endpoints & headless in one breath

- Every Service tracks backend Pods via Endpoints/EndpointSlices
- Normal Service: one ClusterIP load-balances across them
- Headless (`clusterIP: None`): DNS returns every Pod IP directly
- Use headless when identity matters — StatefulSets, peer discovery

HANDS-ON

Now run `Lab 4.4`

`labs/4.4/`

Next: **4.5 — Ingress**